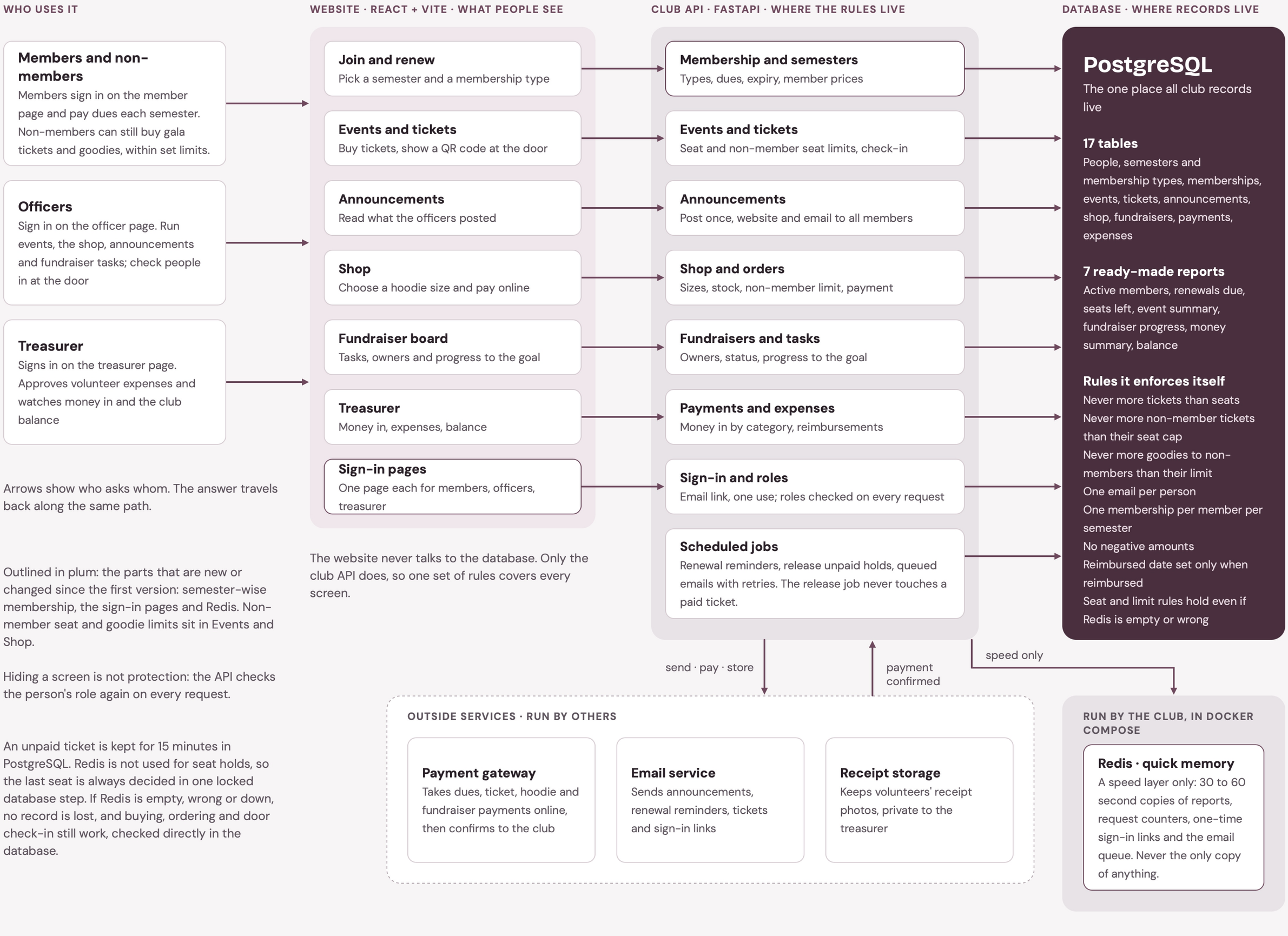
Oddoskyline system architecture

One React website, one FastAPI backend and one PostgreSQL database for the Skyline Student Association, with a Redis quick memory beside them. Each screen matches one scene of the brief, and the rules live in the API and the database so every screen follows them.



Arrows show who asks whom. The answer travels back along the same path.

Outlined in plum: the parts that are new or changed since the first version: semester-wise membership, the sign-in pages and Redis. Non-member seat and goodie limits sit in Events and Shop.

Hiding a screen is not protection: the API checks the person's role again on every request.

An unpaid ticket is kept for 15 minutes in PostgreSQL. Redis is not used for seat holds, so the last seat is always decided in one locked database step. If Redis is empty, wrong or down, no record is lost, and buying, ordering and door check-in still work, checked directly in the database.

The website never talks to the database. Only the club API does, so one set of rules covers every screen.

Oddoskyline data model

17 tables in five groups, plus 7 ready-made reports. New in this version: semesters and membership types, a seat cap for non-members at the gala, a limit on goodies for non-members, and orders that non-members can place.

PK unique id of a row · **FK** points to a row in another table · arrow: "refers to" · **NEW** added in this version · plum outline: table with new or changed columns

People and membership

Members, the semester they paid for, the leadership team and announcements

semesters NEW

A named date range

- `semester_id` PK
- `name` e.g. Fall 2026, unique
- `starts_on`
- `ends_on`

membership_types NEW

What a member can buy

- `membership_type_id` PK
- `name` unique
- `dues_amount` per semester
- `is_offered` stop selling a type

memberships

One member, one semester, one type

- `membership_id` PK
- `member_id` FK
- `semester_id` FK NEW
- `membership_type_id` FK NEW
- `dues_amount` price at purchase
- `payment_id` FK → **payments** empty = unpaid
- `renewal_reminder_sent_at`
- `UNIQUE (member_id, semester_id)`

Replaces valid_from and valid_until

members

Everyone who has signed up

- `member_id` PK
- `full_name`
- `email` one per person
- `phone`
- `joined_on`

announcements

Posted once, shown and emailed

- `announcement_id` PK
- `title`
- `body`
- `posted_by` FK
- `posted_at`

leadership_roles

The elected team, every year

- `leadership_role_id` PK
- `member_id` FK
- `position` e.g. Treasurer
- `term_year`

Semester-wise membership

A semester is a named date range, like Fall 2026. Semesters cannot overlap.

Each membership type sets its own dues for a semester.

A member holds at most one membership per semester.

Active means paid, and today falls inside that semester.

Events and tickets

The Spring Gala: prices, seats for members and non-members, and the door

events

e.g. the Spring Gala

- `event_id` PK
- `title`
- `description`
- `venue`
- `starts_at`

ticket_types

Prices and seats for an event

- `ticket_type_id` PK
- `event_id` FK
- `name`
- `member_price`
- `non_member_price`
- `seat_limit` all seats
- `non_member_seat_limit` NEW

tickets

One row per ticket sold

- `ticket_id` PK
- `ticket_type_id` FK
- `member_id` FK → **members** empty = non-member
- `holder_name`
- `holder_email`
- `price_paid`
- `payment_id` FK → **payments**
- `ticket_code` scanned at the door
- `purchased_at`
- `checked_in_at` empty = not arrived
- `checked_in_by` FK → **members**

Two seat limits on every ticket type

seat_limit is all seats. Every ticket counts, member or not.

non_member_seat_limit is the most seats non-members can hold, never above seat_limit. Zero means members only.

A rule inside the database checks both limits before each sale, so two people buying the last seat at the same moment cannot both get it.

Price follows the ticket holder's email: an active member pays member_price, anyone else pays non_member_price and uses a non-member seat.

Shop

Hoodies and T-shirts: sizes, stock and orders

products

Hoodies, T-shirts and other goodies

- `product_id` PK
- `name`
- `description`
- `price` non-member price
- `member_price`
- `non_member_limit` NEW

product_variants

One row per size

- `variant_id` PK
- `product_id` FK
- `size`
- `stock_quantity`

order_items

The lines of an order

- `order_id` PK FK
- `variant_id` PK FK
- `quantity`
- `unit_price` price at purchase

orders

Placed by members and non-members

- `order_id` PK
- `member_id` FK → **members** empty = non-member
- `buyer_name` NEW
- `buyer_email` NEW
- `ordered_at`
- `status` placed, paid, cancelled
- `payment_id` FK → **payments**

Goodies for non-members

non_member_limit is the most units of a product that non-members can get in total.

Non-member orders are served first come, first served until it runs out. Members are held back only by stock.

Fundraisers

Who does what, and how close the club is to its goal

fundraisers

e.g. a bake sale

- `fundraiser_id` PK
- `title`
- `description`
- `event_date`
- `goal_amount`

fundraiser_tasks

Who is doing what

- `task_id` PK
- `fundraiser_id` FK
- `title`
- `description`
- `assigned_to` FK → **members**
- `status` todo, in progress, done
- `due_date`

Progress against the goal

Money raised is the sum of payments linked to the fundraiser. Tasks done are counted against tasks in total. Both are compared with goal_amount.

Money

Every rupee that comes in, and every volunteer expense paid back

payments

Every rupee that comes in

- `payment_id` PK
- `member_id` FK → **members** empty = non-member
- `category` what it paid for
- `amount` above zero
- `method` cash or online
- `paid_at`
- `fundraiser_id` FK → **fundraisers**

expenses

Volunteer spending, paid back

- `expense_id` PK
- `member_id` FK → **members** the volunteer
- `description`
- `amount`
- `receipt_url`
- `event_id` FK → **events**
- `fundraiser_id` FK → **fundraisers**
- `incurred_on`
- `submitted_at`
- `status` see money board
- `reimbursed_at` set only when paid back

How money is kept

A payment is for one of four things: dues, ticket, merchandise or fundraiser. Cash and online payments sit in the same table.

Memberships, tickets and orders each point to their payment. An empty payment_id means not paid yet.

An expense is submitted, then approved, then reimbursed, or it is rejected. The reimbursed date is set only at the last step.

Club balance = all payments minus reimbursed expenses.

Ready-made reports

Views: the answers each scene asks for, worked out from the tables above

- **active_members** — Who is a member this semester
- **memberships_due_for_renewal** — Who to remind before their semester ends
- **ticket_availability** — Seats left, in total and for non-members
- **event_summary** — Tickets sold, who came, money made
- **fundraiser_progress** — Tasks done and money raised vs goal
- **treasurer_summary** — Money in by category, money out
- **club_balance** — Money in minus reimbursed expenses

Redis keys (temporary, not tables) NEW

Redis is a speed layer only. Nothing in it is a record, and none of it is a table, so the count stays at 17. If Redis is empty, the club loses nothing.

KEY PATTERN	WHAT IT STORES	HOW LONG IT LIVES
`signin:{token}`	One-time sign-in link, deleted on first use	15 min
`rate:{endpoint}:{ip}`	Request counter for sign-in email links, ticket purchase and shop orders	1 min
`cache:{report_name}`	Copy of a report: ticket_availability, active_members, club_balance or fundraiser_progress. Cleared when a payment is confirmed or a ticket is checked in	30 to 60 s
`queue:emails`	Emails waiting to be sent: renewal reminders, announcements, ticket QR codes and sign-in links. Up to 3 attempts, then logged	Until sent

Membership, one semester at a time

A member buys a membership for one semester and picks a type. It counts only while that semester runs, and a reminder goes out before it ends. Names, dates and prices in [brackets] are for the club to decide.

1 What a membership is made of

A member
Anyone who has signed up. Name, email and phone are kept once, not once per semester.

+

A semester
Like Fall 2026 or Spring 2027, with a start date and an end date. Officers add one each term.

+

A membership type
For example Basic, Plus or Premium. Each type has its own dues for a semester.

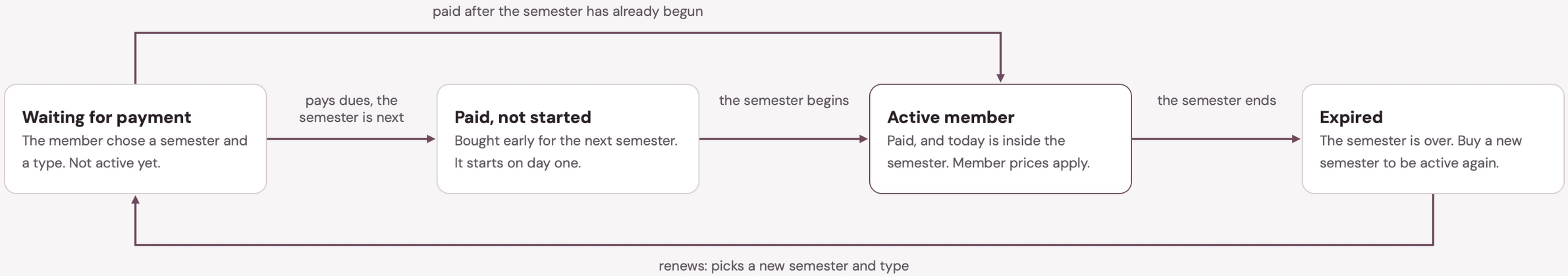
makes →

A membership
One member, one semester, one type. The dues are copied in at purchase, so a later price change never alters old records.

Every type gets the member price on gala tickets and goodies. The brief has a single member price, so a type changes the dues only. Example type names are not final.

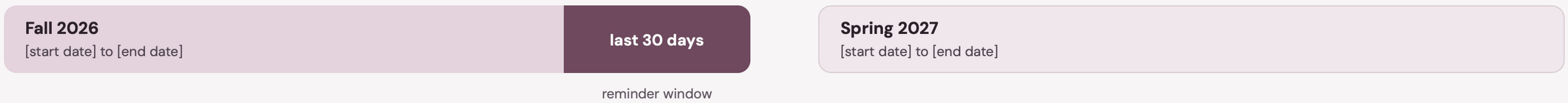
2 Life of one membership

The stage is worked out from two facts: has the dues payment arrived, and where does today fall against the semester dates.



3 Renewal reminder, semester by semester

Reminders follow the end of the semester the member paid for, not a calendar year.



1 Daily check
Finds paid members whose semester ends within 30 days and who have not bought the next one.

→

2 One reminder email
Sent once per membership. The send time is saved, so nobody gets two.

→

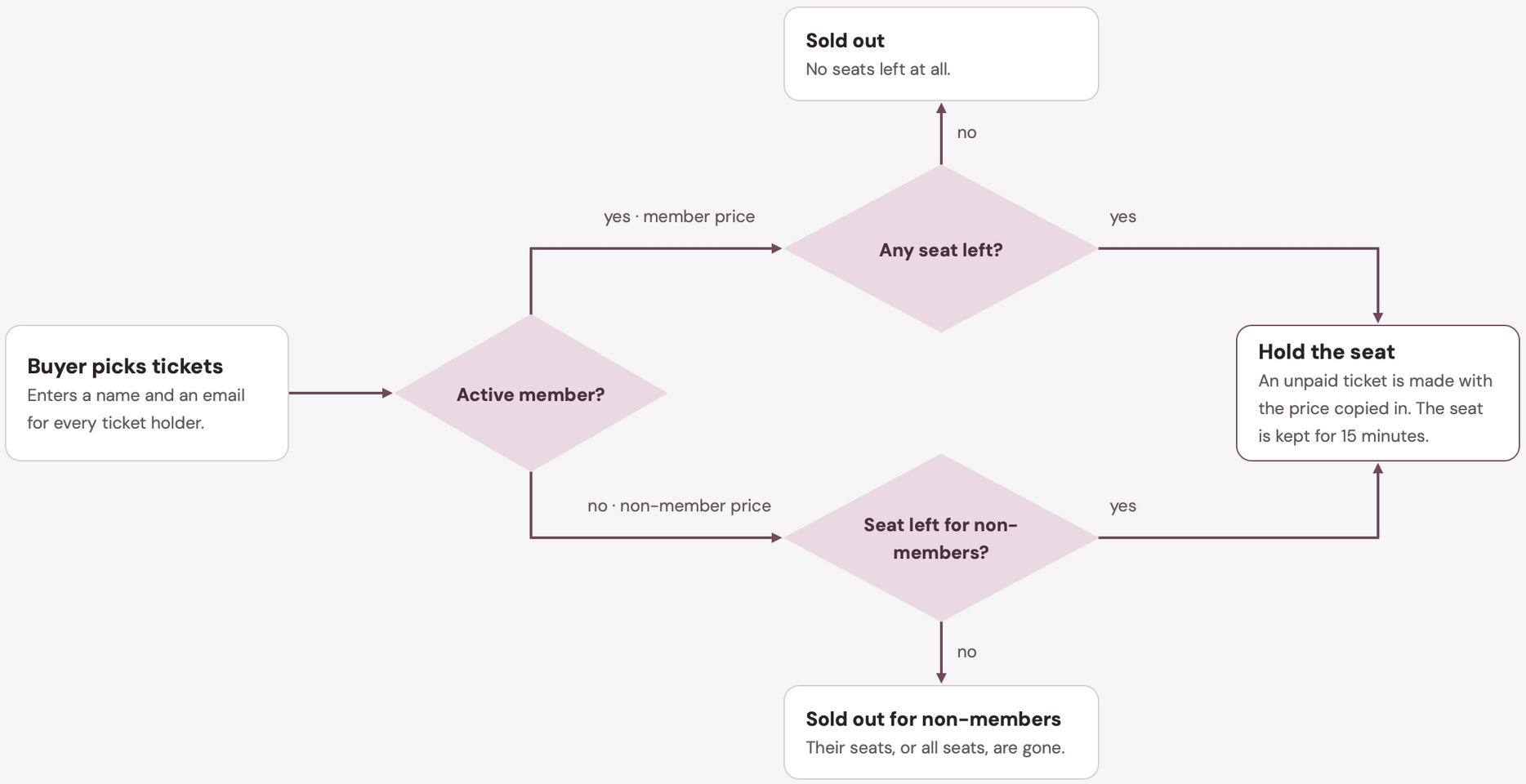
3 Member renews
Picks the next semester and a type, pays dues, and is Paid, not started until day one.

Spring Gala: tickets and door check-in

Members and non-members can both buy tickets. Non-members have their own, smaller seat limit inside the total. A seat is held for 15 minutes while the buyer pays.

1 Choosing and holding seats

Price and seat type follow each ticket holder, not the buyer. A member buying for friends pays the non-member price for them.



Why there are two checks

Members can take any seat that is left.

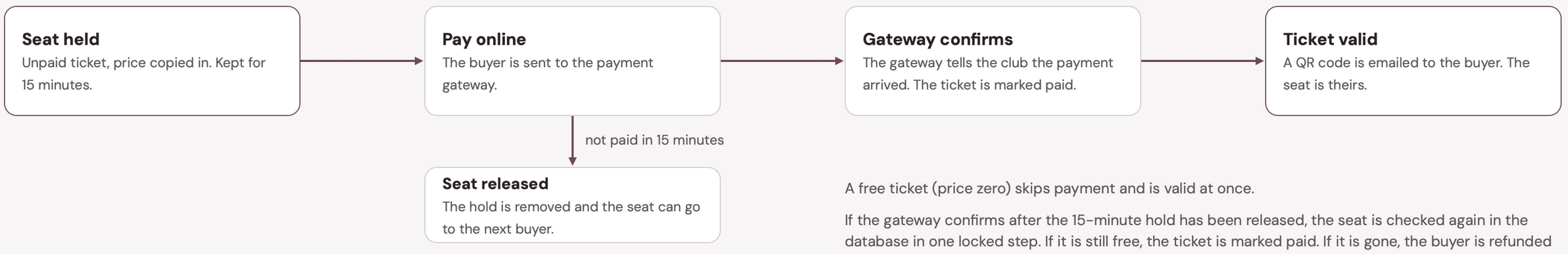
Non-members can take only the seats set aside for them, and only while seats are left overall.

Both checks run in one locked step, so two buyers cannot get the last seat.

Redis is not used for seat holds, so the last seat is always decided in one locked database step.

2 Paying and getting the ticket

Only the payment gateway's confirmation marks a ticket as paid, never the buyer's browser.

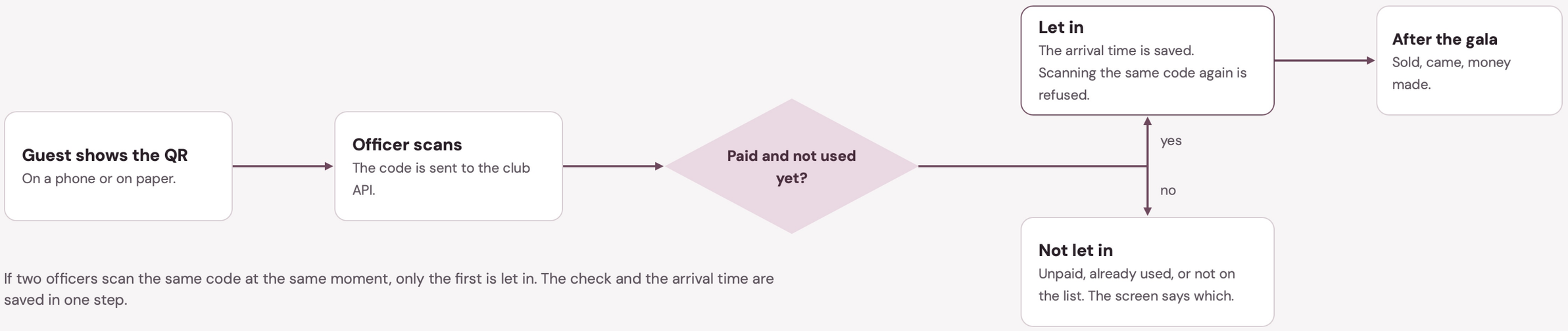


A free ticket (price zero) skips payment and is valid at once.

If the gateway confirms after the 15-minute hold has been released, the seat is checked again in the database in one locked step. If it is still free, the ticket is marked paid. If it is gone, the buyer is refunded and told.

3 At the door

An officer scans the QR code from the ticket email, or types the code in.



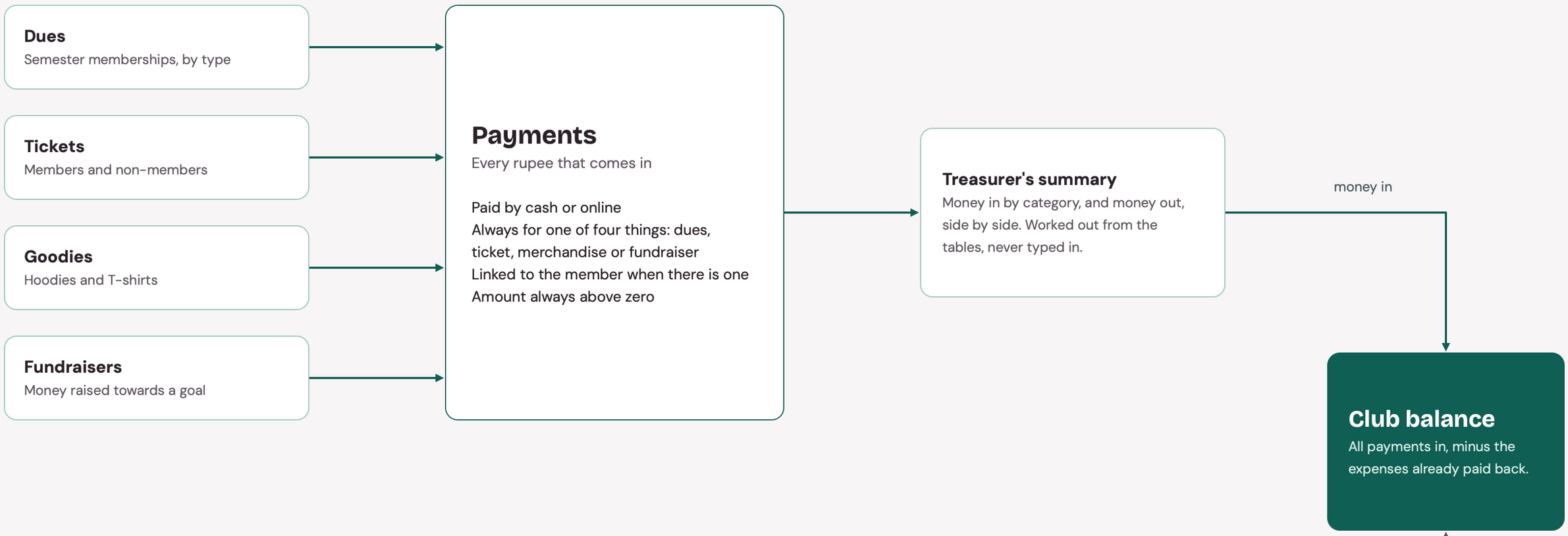
If two officers scan the same code at the same moment, only the first is let in. The check and the arrival time are saved in one step.

Money in, money out

Every rupee that comes in is a payment. Every rupee that goes out is a volunteer expense paid back. The club balance is the difference, so the treasurer never has to add up a notebook.

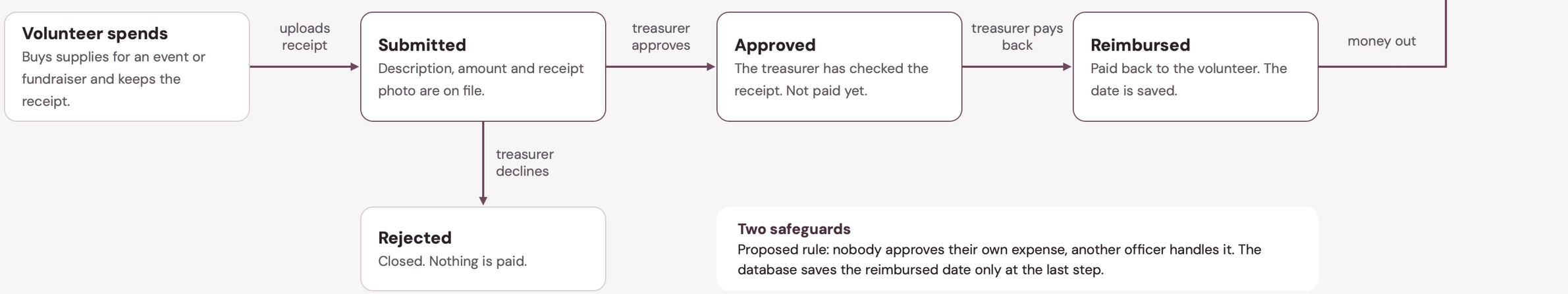
1 Money in

Cash and online payments sit in the same table, so the treasurer sees one list.



2 Money out: paying volunteers back

A volunteer's own money is paid back only after the treasurer has checked the receipt.



Who can do what

Four kinds of people use Oddoskyline. What each can do follows the six scenes in the brief. Tinted rows are new or changed: semester memberships, the non-member limits and the sign-in pages.

Action	Anyone Non-member or visitor	Member Signed up; member prices while this semester is paid	Officer On this year's leadership list	Treasurer The officer holding that position
Before the scenes · Signing in				
Which sign-in page they use (email link, works once)	None; browsing needs no sign-in	Member page	Officer page	Treasurer page
Areas they can open	Public pages	Public pages and My stuff	Adds Manage and Finance	Adds the Treasurer dashboard
Signs in on a page for a role they do not hold	Not applicable	Stays a member, sees "No access"	Treasurer page: "No access"	Can open all three areas
Scene 1 · A new student joins				
Sign up and see your own membership	Sign up	Own	Everyone's	Everyone's
Pay dues for a semester	No	Own, online	Anyone's, cash or online	Anyone's, cash or online
Set up semesters and membership types (proposed)	No	No	Yes	Yes
Scene 2 · Spring Gala tickets				
Buy tickets	Yes, non-member price, limited seats	Yes, member price	Yes	Yes
Set ticket prices, seats and the non-member seat limit	No	No	Yes	Yes
Check people in at the door	No	No	Yes	Yes
Scene 3 · Announcements				
Read announcements	Yes	Yes	Yes	Yes
Post an announcement	No	No	Yes	Yes
Scene 4 · Hoodies and T-shirts				
Order goodies	Yes, non-member price, limited quantity	Yes, member price	Yes	Yes
Manage products, sizes, stock and the non-member limit	No	No	Yes	Yes
Scene 5 · Fundraiser				
See tasks and progress to the goal	No	Yes	Yes	Yes
Update a task's status	No	Own tasks	Any task	Any task
Create fundraisers and assign tasks	No	No	Yes	Yes
Scene 6 · Treasurer				
Submit an expense with a receipt	No	Own	Own	Own
See money in, expenses and the balance	No	No	Yes	Yes
Approve, reject and reimburse expenses	No	No	No	Yes, never their own

Roles are checked on the server for every request, never trusted from the screen or from the sign-in page used; the page only decides where a person lands. Non-members who buy tickets or goodies give a name and an email, and pay online. The row marked proposed is not in the brief.